

Контролно по ООП-практикум. 30.05.2026

В тази задача ще имплементирате софтуерна имитация на мрежов сървър, който управлява активни връзки с клиентски устройства от различен тип. Времето в системата ще се измерва в тактове, които сървърът ще отброява. Реализирайте следните класове:

Абстрактен клас Device (5 т)

Всяко устройство получава **уникален целочислен идентификатор (ID)** в момента на създаване. Идентификаторът е константен и не се преизползва между инстанции. Устройството има **име** с произволна дължина, което може да е празно.

Наред с методите за достъп до данните всеки вид устройство трябва да поддържа също и следните операции:

- Връщане на низ, идентифициращ конкретния тип ("mobile", "desktop").
- Отпечатване на специфична за типа информация на стандартния изход.

Имплементирайте **два конкретни класа, които са вид Device**:

MobileDevice (1 т) - Съдържа допълнително **ниво на батерия** (цяло число, 0–100). При отпечатване показва името, ID-то и нивото на батерията. Идентифицира се като "mobile".

DesktopDevice (2 т) - Съдържа допълнително поле: **URL адрес** (низ с произволна дължина, не може да е празен). При отпечатване показва името, ID-то и URL адреса. Идентифицира се като "desktop".

Клас Connection (2 т)

Представя информацията за успешна сесия за комуникация между устройство и сървър. Трябва да има информация за устройството, с което е установена и момента (такта), в който това се е случило. Обектът е отговорен за управлението на паметта, свързана с устройството, и трябва да може да се създава само от сървър. Дефинирайте конструктор, който създава връзка по подадено устройство и конкретен момент (такт) на свързване. Класът Connection трябва да поддържа коректно копиране.

Клас Server (8 т)

Централен клас, управляващ определен брой активни сесии. Връзките се осъществяват от сървъра и живеят определено време (timeout), след което трябва да се прекратяват автоматично.

За един сървър трябва да се поддържат поне следните методи:

- `Server(unsigned capacity, unsigned timeout)` — задава максимален брой едновременни връзки и праг на изтичане в тактове. В началото времето е 0.
- `connect(const Device& d)` — добавя нова връзка с подаденото устройство. Ако сървърът е пълен, устройството се отхвърля. Връща дали свързването е успешно.
- `disconnect(int deviceID)` — премахва връзката с даденото ID и **сгъстява масива** (в него не трябва да има дупки).
- `tick(unsigned cnt = 1)` — увеличава текущото време с подадения брой тактове и прекъсва всички връзки, за които е достигнат прагът на изтичане (timeout).
- `saveLog(const char* filename)` — записва текущото състояние на сървъра във файл.
- `loadLog(const char* filename)` — заменя текущото състояние с данните от файла. Устройствата се възстановяват като инстанции на коректните си класове и запазват идентификаторите, записани във файла

Форматът на лог файла е текстов. За всяка активна връзка се записват:

<device_type> <device_id> <device_name> <type_specific_fields> <connected_at>

Примерни редове:

mobile 1 Phone1 87 0

desktop 2 PC1 192.168.0.1 0

Обект от този клас не трябва да се копира.

Демонстрирайте следния сценарий (2 т):

1. Създайте сървър с капацитет **4 връзки** и таймаут **10**.
2. Създайте **6 устройства** от различни типове и с различни имена (поне по едно от всеки тип).
3. **Такт 0**: свържете 2 устройства. **Такт 3**: свържете трето. Запишете `server.log`.
4. **Такт 5**: свържете четвърто устройство. Опитайте да свържете пето — трябва да бъде отхвърлено, тъй като сървърът е пълен.
5. **Такт 11**: Прекъснете ръчно едно от останалите свързани устройства. Свържете петото устройство.
6. Създайте **нов сървърен обект** и заредете `server.log` в него. Свържете шестото устройство към него.
7. Извикайте операцията за отпечатване за всяко устройство в новия сървър, за да демонстрирате полиморфизма.
8. Запишете финален лог и го отпечатайте на екрана.

Общи изисквания.

- За всички класове трябва да имате подходящи методи за достъп до данните.
- Спазвайте ООП принципите. Валидирайте всички данни за вашите класове.
- Подсигурете коректен жизнен цикъл за всички класове. Ако не е посочено друго, се предполага да поддържате копиране на обектите.
- Трябва да предоставите поне слаба гаранция при възникване на изключения.
- Можете да предполагате коректен формат на данните, въвеждани от потребителите и във файла. Нямате гаранция за наличие на файловете и тяхната физическа цялост.
- Можете да използвате библиотеките от езика C, както и стандартните за C++ библиотеки за работа с входно-изходни операции, изключения и памет